



**Instituto:** Tecnológico de Lerdo

**Carrera:** Ingeniería Informática

**Docente:** Jesús Salas Marín

**Materia:** Administración y organización de datos

**Alumnos:**

Jesús Emmanuel Arreola López - 242310336

Jahaziel isaí Fraire Olivares - 242310878

Alexander Adonaí Molina Rodriguez - 242310131

José Angel Torres Cortez - 242310897

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Introducción</b>                             | <b>2</b>  |
| <b>2. Objetivo General</b>                         | <b>2</b>  |
| <b>3. Hipótesis Experimental</b>                   | <b>3</b>  |
| <b>4. Metodología Experimental</b>                 | <b>3</b>  |
| 4.1 Herramientas Utilizadas                        | 3         |
| 4.2 Generación de Datos                            | 3         |
| 4.3 Variables del Experimento                      | 4         |
| Variables Independientes                           | 4         |
| Variables Dependientes                             | 4         |
| <b>5. Desarrollo del Sistema</b>                   | <b>5</b>  |
| 5.1 Generación de Archivos                         | 5         |
| Variables Independientes                           | 5         |
| Variables Dependientes                             | 5         |
| 5.2 Escritura de Archivos                          | 6         |
| 5.3 Lectura de Archivos                            | 6         |
| 5.4 Estadísticas Básicas                           | 6         |
| 5.5 Búsqueda de Pacientes                          | 7         |
| <b>6. Experimentación</b>                          | <b>7</b>  |
| <b>7. Resultados</b>                               | <b>8</b>  |
| <b>8. Visualización de Datos</b>                   | <b>8</b>  |
| 8.1 Gráfica de Barras                              | 8         |
| 8.2 Gráfica de Líneas                              | 9         |
| 8.3 Gráfica Circular                               | 9         |
| 8.4 Histograma                                     | 9         |
| <b>9. Comparación entre CSV y JSON</b>             | <b>9</b>  |
| <b>10. Evaluación del Sistema</b>                  | <b>10</b> |
| ¿Qué organización fue más eficiente?               | 10        |
| ¿Cuál fue más flexible?                            | 10        |
| ¿Cuál facilita más el análisis de datos?           | 10        |
| ¿Cuál consume menos almacenamiento?                | 10        |
| ¿Qué formato recomendarían para hospitales reales? | 10        |
| <b>11. Conclusiones</b>                            | <b>11</b> |

## 1. Introducción

Hoy en día, los hospitales se enfrentan al reto de gestionar cantidades gigantescas de información médica: desde los datos básicos de un paciente y sus consultas, hasta el control de signos vitales, diagnósticos y recetas. Con semejante volumen de información creciendo a diario, elegir cómo guardar y organizar estos archivos ya no es un simple detalle técnico; es una decisión crucial para que el sistema responda rápido, no se sature y ayude a los médicos a trabajar sin trabas.

En el mundo de la organización de datos, los formatos CSV (*Comma-Separated Values*) y JSON (*JavaScript Object Notation*) son las dos herramientas más usadas para almacenar información. El formato CSV es famoso por ser plano, directo y rapidísimo para leer grandes bloques de datos en fila. Por otro lado, JSON brilla por su flexibilidad, ya que permite crear estructuras jerárquicas (como carpetas dentro de carpetas) para guardar datos más complejos de forma natural.

Para este proyecto, diseñamos y programamos un sistema experimental en Python que simula el entorno de un hospital. El objetivo real es poner a competir a ambos formatos bajo condiciones de alta presión, midiendo con precisión cuánto tardan en leer y escribir, cuánto espacio ocupan en el disco duro y qué tan fáciles se lo ponen a un desarrollador a la hora de analizar la información.

## 2. Objetivo General

Desarrollar un sistema funcional en Python que genere, almacene y procese registros médicos masivos utilizando archivos CSV y JSON, con el fin de medir científicamente su rendimiento en base a tiempos de ejecución, espacio en disco y facilidad para crear análisis estadísticos y gráficas visuales.

### 3. Hipótesis Experimental

Nuestra apuesta inicial es que el formato CSV va a ser el más rápido a la hora de escribir y cargar datos masivos, además de que los archivos van a pesar mucho menos en el disco duro. Esto se debe a que es un formato muy simple y plano, sin etiquetas repetitivas.

Por el contrario, esperamos que JSON demuestre ser mucho más flexible. Al permitir guardar datos anidados (como meter los signos vitales y los detalles de la consulta dentro del mismo registro del paciente), creemos que JSON se perfila como la mejor opción para conectar diferentes sistemas médicos modernos o trabajar con aplicaciones web (APIs) que necesitan mover datos complejos sin romper la estructura.

### 4. Metodología Experimental

#### 4.1 Herramientas Utilizadas

|                        |                                                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Herramienta / Librería | tilizarán para leer y escribir archivos CSV y JSON, otras servirán para medir el tiempo de ejecución y el rendimiento del sistema |
| Python 3.x             | El lenguaje base en el que programamos toda la lógica del sistema y el menú interactivo.                                          |
| Módulo csv             | Lo usamos para escribir los datos fila por fila desde cero, evitando que la computadora se quede sin memoria.                     |

|                            |                                                                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Módulo json</b>         | <b>Nos permitió empaquetar y estructurar la información médica compleja en formato de árbol.</b>                                              |
| <b>Pandas</b>              | <b>Es el motor que mueve el menú del programa; lo usamos para cargar los datos al instante, hacer estadísticas y buscar pacientes rápido.</b> |
| <b>Matplotlib (pyplot)</b> | <b>La librería encargada de transformar los números abstractos en las 4 gráficas que pide el proyecto.</b>                                    |
| <b>Random</b>              | <b>Para inventar de forma aleatoria nombres, edades, diagnósticos y medicamentos dentro de rangos médicos lógicos.</b>                        |
| <b>Time</b>                | <b>Nuestro cronómetro de alta precisión. Captura los tiempos de carga en microsegundos.</b>                                                   |
| <b>OS / Datetime</b>       | <b>Con <i>OS</i> medimos el peso exacto de los archivos en bytes, y con <i>Datetime</i> ponemos la fecha y hora real a los reportes.</b>      |
| <b>Tabulate</b>            | <b>Una pequeña librería para que la tabla comparativa de resultados se vea ordenada y estética en la consola.</b>                             |

## 4.2 Generación de Datos

Para que el experimento fuera realista, creamos un motor que genera registros simulados. Cada paciente cuenta con 9 campos bien definidos:

1. **id:** El número de identificación único del paciente (autoincremental).

2. Nombre y apellido combinado al azar desde nuestra base de datos interna.
3. edad: Un número entero que va desde 1 hasta los 90 años.
4. mes: El mes en el que se hizo la consulta (clave para ver cómo se comporta el hospital a lo largo del año).
5. temperatura: Un valor decimal que simula la realidad (entre 36.0°C y 39.5°C).
6. Presión: Texto que representa la presión arterial (por ejemplo, "120/80").
7. medicamento: El fármaco recetado (Paracetamol, Ibuprofeno, Amoxicilina, Omeprazol o Metformina).
8. especialidad: El área médica que atendió al paciente (Cardiología, Pediatría, General, Urgencias o Ginecología).
9. Enfermedad: El diagnóstico final asignado (Gripe, Diabetes, Hipertensión, Covid o Migraña).

## 4.3 Variables del Experimento

**Variables Independientes:** El formato de archivo utilizado (CSV plano vs. JSON jerárquico) y la cantidad de registros evaluados en las pruebas de estrés (1 millón, 10 millones y 20 millones de filas).

**Variables Dependientes:** El tiempo que tarda el programa en escribir los archivos, el tiempo que tarda en leerlos, el peso final de los archivos en el disco duro y la velocidad de respuesta al buscar datos filtrados.

## 5. Desarrollo del Sistema

### 5.1 Generación de Archivos

Para facilitar las cosas, todas las variables clave del programa se manejan desde el archivo `config.py`. Si el profesor quiere probar el sistema con 1 millón, 10 millones o 20 millones de datos, sólo hay que cambiar el número en la variable `NUM_REGISTROS` y el programa se adapta automáticamente en la siguiente ejecución.

### 5.2 Escritura de Archivos

Cuando manejas millones de registros, es muy fácil congelar una computadora si intentas cargar todo a la vez en la memoria RAM. Para evitar esto, programamos el sistema usando "flujos de escritura" (*Streams*) con la instrucción `with open()`.

- En CSV: Usamos `csv.writer()` para ir guardando fila por fila directamente en el disco duro. Así, la memoria RAM se mantiene libre y ligera, sin importar si generamos 1 o 20 millones de registros.
- En JSON: Optamos por un formateo iterativo que serializa los datos sobre la marcha para agilizar el proceso y amortiguar el peso de las comillas y llaves de su sintaxis.

## 5.3 Lectura de Archivos

Para cumplir con la rúbrica y a la vez hacer un programa que funcione bien en la vida real, dividimos la lectura de datos en dos estrategias:

1. Para el Experimento (Al arrancar): El programa abre los archivos usando los módulos nativos de Python (`csv.reader` y el lector de `json`) para leer los datos de forma pura, línea por línea. Esto nos permite medir el rendimiento real del formato sin ayudas externas y guardar los resultados en un archivo llamado `comparativa_resultados.txt`.
2. Para el Menú del Usuario (En tiempo real): Una vez que arranca el menú interactivo, el sistema pasa a utilizar Pandas a través de la función `actualizar_dataframe()`. Pandas es increíblemente rápido cargando archivos CSV (`pd.read_csv()`), lo que permite que el usuario busque pacientes o vea estadísticas al instante, sin esperas molestas.

## 5.4 Estadísticas Básicas

Gracias al uso de Pandas, el sistema puede procesar esos millones de datos en memoria en un pestañeo. Al elegir la opción de estadísticas, calcula al instante el promedio de temperatura de los pacientes, la edad máxima registrada y qué enfermedades o especialidades son las más comunes en el hospital.

## 5.5 Búsqueda de Pacientes

El programa permite buscar pacientes de dos maneras: escribiendo su ID exacto o simplemente tecleando una parte de su nombre (usando la función interna `.str.contains()` de Pandas, que ignora si escribes con mayúsculas o minúsculas).

¿Qué pasa al agregar un paciente nuevo? Para evitar que el usuario meta información errónea, programamos validaciones estrictas en `funciones.py`. El

sistema no te dejará guardar un paciente si dejas el nombre en blanco, si pones una edad irreal (como menos de 0 o más de 120 años) o si la temperatura no tiene sentido biológico. Si el paciente pasa los filtros, el programa abre el archivo CSV y el JSON al mismo tiempo en modo de añadir (*Append "a"*), guardando los datos en ambos repositorios para que las dos bases de datos se mantengan idénticas y actualizadas.

## 6. Experimentación

La fase de pruebas consistió en correr el programa cambiando la configuración a las tres escalas requeridas (1M, 10M y 20M de registros). Durante cada prueba, dejamos que el software midiera con su cronómetro interno los tiempos de entrada y salida de datos (I/O) y revisamos el tamaño final de los archivos generados en la carpeta del proyecto.

## 7. Resultados

Luego de completar las pruebas con millones de datos, los números nos dieron la razón en varios puntos clave:

- El formato CSV destrozó a JSON en velocidad; fue muchísimo más rápido escribiendo los datos en el disco y cargándolos en la memoria al principio.
- En cuanto a almacenamiento, el CSV ocupó un espacio ridículamente menor que JSON. Al no tener que repetir las etiquetas en cada renglón, los archivos resultaron muy ligeros.
- La combinación de CSV + Pandas demostró ser la fórmula perfecta para hacer análisis numérico rápido sin que el procesador de la computadora sufra.
- Por su parte, JSON demostró su valor en la parte organizativa. Guardar la información en categorías anidadas hizo que el archivo fuera mucho más fácil de entender para un humano y demostró que se adapta a cualquier cambio de estructura sin despeinarse.

## 8. Visualización de Datos

Para la representación gráfica de la información se utilizó la librería Matplotlib.

El sistema generó diferentes tipos de gráficas para facilitar el análisis visual.



## 8.1 Gráfica de Barras

La gráfica de barras permitió visualizar la cantidad de consultas registradas por especialidad médica.

Gracias a esta representación fue posible identificar las áreas con mayor demanda hospitalaria.

## 8.2 Gráfica de Líneas

La gráfica de líneas mostró la evolución mensual de la actividad hospitalaria.

Esto permitió identificar tendencias y periodos con mayor cantidad de consultas.

## 8.3 Gráfica Circular

La gráfica circular representó la distribución de medicamentos recetados.

Con esta información se pudieron identificar los medicamentos más utilizados dentro del hospital.

## 8.4 Histograma

El histograma mostró la distribución de pacientes por rango de edad.

Esta gráfica permitió analizar qué grupos de edad fueron más frecuentes dentro del sistema hospitalario.

## 9. Comparación entre CSV y JSON

### Comparación Avanzada entre CSV y JSON

| Característica          | Organización en CSV                                                                    | Organización en JSON                                                                                 |
|-------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Velocidad de Lectura    | Ultra rápida. Estructura plana ideal para procesar millones de filas en cadena.        | Media-Baja. El sistema tarda más porque debe revisar llaves y corchetes uno por uno.                 |
| Velocidad de Escritura  | Máxima eficiencia. Guarda los datos directamente en línea sin dar rodeos.              | Moderada. El procesador trabaja más dando formato al texto y ordenando las etiquetas.                |
| Espacio en Disco        | Excelente (Muy ligero). Guarda sólo los datos puros separados por comas.               | Pesado. Consume mucho espacio porque repite los nombres de los campos en cada renglón.               |
| Flexibilidad de Datos   | Rígido. Si añades un dato nuevo, te obliga a cambiar las columnas de toda la tabla.    | Espectacular. Puedes meter campos nuevos o anidados en un paciente sin romper los demás.             |
| Uso en el Mundo Moderno | Limitado. Se usa más para respaldos, hojas de cálculo o análisis estadísticos locales. | Líder absoluto. Es el idioma estándar para enviar datos por internet y conectar aplicaciones (APIs). |

|                     |                                                                              |                                                                                                 |
|---------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Conexión con Pandas | Perfecta. Se transforma en una tabla de Pandas de forma directa e inmediata. | Aceptable. A veces requiere pasos extra para "aplanar" los datos anidados antes de analizarlos. |
|---------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|

## 10. Evaluación del Sistema

¿Qué organización fue más eficiente? A nivel de rendimiento técnico puro, el formato CSV ganó por goleada. Fue el más rápido procesando datos y el que menos espacio le robó al disco duro.

¿Cuál demostró ser más flexible? El formato JSON se llevó este punto sin discusión. Su capacidad de crear estructuras jerárquicas complejas permite diseñar expedientes médicos mucho más ricos y detallados.

¿Cuál facilita más el análisis de datos? El formato CSV, debido a lo bien que se integra con la librería Pandas para hacer cálculos matemáticos y estadísticos en tiempo récord.

¿Qué formato recomendarían para un hospital real? Para un hospital de verdad, nuestra recomendación formal es usar una arquitectura híbrida. No hay por qué elegir sólo uno si puedes tener lo mejor de ambos mundos:

1. Se debe usar CSV para las bases de datos históricas y los sistemas de analítica del hospital (*Business Intelligence*). Al ser tan ligero y rápido con Pandas, es perfecto para revisar estadísticas de años anteriores o patrones de enfermedades.
2. Se debe usar JSON para el software del día a día y la comunicación. Es el formato ideal para conectar el sistema del hospital con laboratorios externos, enviar expedientes a otros hospitales o actualizar las aplicaciones de las enfermeras en tiempo real a través de APIs web.

## 11. Conclusiones

Trabajar en este proyecto nos abrió los ojos sobre la importancia de la materia de Administración y Organización de Datos. Nos demostró que la forma en que decides guardar los archivos define por completo qué tan rápido, escalable y eficiente va a ser un sistema informático en el mundo real.

Gracias a las pruebas de estrés que hicimos con millones de registros, aprendimos que en la ingeniería informática no existen "formatos mejores que otros" de forma absoluta; todo depende de lo que necesite tu negocio. Mientras que el formato plano CSV es el rey indiscutible de la velocidad y el ahorro de espacio cuando necesitas analizar datos masivos en frío, el esquema jerárquico de JSON justifica plenamente su peso y su consumo de procesador al ofrecer una flexibilidad y una capacidad de adaptación que los formatos tradicionales simplemente no pueden alcanzar.